AI Resume Screening & Ranking System

Engineering Architecture, Pipeline Steps, and Execution Guide

1. Executive Overview & Goals

A production-minded AI Resume Screening & Ranking System designed for evaluating candidates for an **SDE Intern (AI/Agentic Systems)** role. Prioritizing **engineering judgment, explainability, correctness, and speed**, it processes batches of resumes, applies strict rule-based eligibility filtering, enriches profiles with GitHub telemetry, and scores eligible candidates against a 100-point rubric with actionable reasoning.

Component	Technology / Implementation	Key Responsibility
Language & Runtime	Python 3.12, Asyncio	High throughput, non-blocking concurrent batch execution
PDF Parser	pypdf (Multi-page)	Robust text & contact extraction, malformed error-tolerant
Eligibility Filter	Rule-Based Regex Engine	Zero-token pre-filter for Python & AI requirements
GitHub Telemetry	GitHub REST API + TTL Cache	Velocity & repo signals (0-10 pts), rate-limit tolerant
Scoring Engine	100-pt Rubric + Penalties	Deterministic baseline + Gemini structured output
Interfaces	CLI + FastAPI REST API	main.py CLI and interactive Swagger API endpoints

2. Pipeline Execution Steps

Step 1: Resume Ingestion & Normalization (`src/extractor.py`)

- Concurrently ingests all PDFs from the input directory.
  - Concatenates text across multi-page layouts and extracts candidate name (handling multi-column Canva layouts & merged lines), email, phone, GitHub profile, LinkedIn, and skills.
  - *Fault Tolerance:* Malformed or unreadable files are skipped cleanly without halting the batch run.

Step 2: Hard Eligibility Rule-Based Filter (`src/filters.py`)

- **Python Evidence (Condition 1):** Requires explicit Python stack keywords (Python, FastAPI, Django, Flask, PyTorch, Pandas, Asyncio). Non-Python profiles are rejected.
- **AI / Agentic Evidence (Condition 2):** Requires meaningful AI project exposure (LangChain, LangGraph, RAG, Vector Search, Tool-Calling, ChromaDB, LlamaIndex).
- **Permissive Stack Policy:** JavaScript, React, or Next.js do *not* trigger rejection if Python + AI are satisfied.
- *Efficiency:* Executes deterministically outside the LLM to eliminate token waste and prevent hallucinations.

Step 3: GitHub Telemetry & In-Memory Caching (`src/github.py`)

- Queries GitHub REST API (`/users/{username}` & `/repos`) with TTL cache.
  - Scores activity up to 10 pts (0–5 pts for recent updates + 0–5 pts for maintained Python/AI repositories and stars).
  - *Graceful Fallback:* Rate limits (HTTP 403), missing profiles, or offline networks record a note without crashing.

Step 4: 100-Point Candidate Scoring & Penalties (`src/scoring.py`)

- **AI Project Depth (40 pts):** Agents, LangGraph, RAG, embeddings, vector databases, tool-calling.
- **Python & Backend (30 pts):** FastAPI, Asyncio, PostgreSQL, Redis, database modeling.
- **Cloud & Full Stack (15 pts):** GCP, Docker containers, CI/CD, React/Next.js supporting frontend.
- **GitHub Activity (10 pts):** Velocity and repository maintenance signals.
- **Engineering Depth (5 pts):** Pytest automated testing, clean architecture, message queues, observability.
- **Project Quality Penalties (-5 to -15 pts):** Deductions for shallow API wrappers (e.g. basic prompt + Streamlit) and tutorial clones.

Step 5: Ranking & Explainable Output (`src/pipeline.py`)

- Eligible candidates are sorted descending by total score and assigned 1-indexed ranks (1, 2, 3...).
  - Rejected candidates receive rank `null` and explicit rejection reasons.
  - Produces structured JSON output, CSV spreadsheet export, and rich terminal summary table.

3. How to Run & Output Commands

The system provides both a high-performance CLI and an interactive FastAPI REST interface.

1. Standard Batch Screening (JSON Output)

```
python main.py --input ./resumes --output ./output/results.json
```

Ingests all resumes in `./resumes`, scores candidates, prints a formatted terminal summary table, and writes structured JSON to `./output/results.json`.

2. Dual Export (JSON + CSV Spreadsheet)

```
python main.py --input ./resumes --output ./output/results.json --csv ./output/results.csv
```

Generates both machine-readable JSON results and a clean spreadsheet CSV containing ranks, scores, and rejection reasons.

3. Pure Deterministic Offline Mode

```
python main.py --input ./resumes --output ./output/results.json --no-llm
```

Runs 100% locally with zero external API calls or token consumption.

4. Start FastAPI Web Service

```
python main.py --serve --port 8000
```

Boots the Uvicorn web server at `http://localhost:8000`. Interactive Swagger docs available at `http://localhost:8000/docs`.

5. Run Automated Test Suite

```
pytest -v
```

Runs all 19 unit and integration tests across extraction, filters, scoring, pipeline, and FastAPI endpoints.

4. Batch Results Summary (50 Resumes)

Metric	Count	Percentage	Notes
Total Ingested	50	100%	All PDF resumes processed in batch
Successfully Parsed	50	100%	Zero unreadable / crashed files
Passed Eligibility	34	68%	Satisfied Python + AI project conditions
Rejected Candidates	16	32%	Non-Python or non-AI stack profiles

Top 6 Ranked Shortlisted Candidates:

Rank	Candidate	Score	AI	Py	Cld	GH	Eng	Source File
1	V Sree Raghu Vardhan	**87.0**	40.0	30.0	13.0	2.0	2.0	candidate_30.pdf
2	Yash Maini	**85.0**	40.0	27.0	15.0	2.0	1.0	candidate_13.pdf
3	Hari Shanker Sharma	**85.0**	36.0	30.0	13.0	2.0	4.0	candidate_41.pdf
4	Arjun Kumar	**83.0**	40.0	27.0	13.0	2.0	1.0	candidate_08.pdf
5	Vaibhav Wakde	**82.0**	40.0	26.0	9.0	2.0	5.0	candidate_33.pdf
6	Arunima Saha	**81.0**	40.0	21.0	15.0	2.0	3.0	candidate_43.pdf